

8

Maskable Interrupts

8.1 Objectives:

In lab 8, we used the polling technique for parallel I/O. Using interrupts can improve performance of the computer and make the software easier to organize. In this lab, you will learn:

- What is an interrupt?
- The difference between maskable and non-maskable interrupts.
- Response of Tiva to a maskable interrupt.
- How to generate delays using the System Timer.
- How to write interrupt service routines (ISR) for Tiva.

8.2 Related material to read:

- Interrupts: Chapter 11 in the text, pages 237 – 268.
- TM4C123GH6PM Data Sheet, Sections 3.3 and 3.4.
- Valvano Volume 1, Section 4.4 and Chapter 9.
- setup.s: for a complete Interrupt Vector Map

8.3 Maskable and Non-Maskable Interrupts:

The Tiva register model was introduced early in the semester. This register model includes the **Program Status Register (PSR)**. Eight of the bits in the IPSR (bits 7:0) indicate which interrupt, if any, is currently being handled.

Another special register associated with interrupts is the **PRIMASK** register. The PRIMASK register contains only one bit (also named PRIMASK), which enables or disables maskable interrupts. The user cannot turn off a non-maskable interrupt.

A maskable interrupt may be turned on or off by the user using several assembly instructions. Its corresponding interrupt masking bit (I) is set to logic 1 during system reset, which turns off the maskable interrupt system. The user can turn on this system using the **Change Processor State Interrupt Enable CPSIE I** instruction and turn off this system using the **Change Processor State Interrupt Disable CPSID I** instruction. Tiva is equipped with several maskable interrupts including Timer, Pulse Accumulator, SPI, and A/D system. In this lab, we will consider only one maskable interrupt, the System Timer (SysTick).

The System Timer generates an interrupt at a user-specified interval. This interrupt is very useful in reminding Tiva to perform regular, repetitive tasks such as measuring the temperature in a room at a regular interval or to switch between multiple concurrently executing processes in a multi-process system.

A complete interrupt vector map for Tiva is listed in the file setup.s available in the zip file for this lab. This map tells you where the interrupt vector of the appropriate interrupt service routine is located for all possible Tiva interrupts.

8.4 Interrupt Service Routine:

An *interrupt service routine or ISR* is similar to a subroutine. Like most programs, the main program may run in an infinite loop to do some useful work. When a module is ready to be serviced, it sets a flag, this causes the main program to turn the control over to the appropriate ISR (the ISR for that particular module). The ISR services the I/O device by clearing the flag, doing data transfers, and any other work required. An ISR is similar to a subroutine but it responds to a hardware signal whereas a subroutine is called by an instruction in the program.

8.5 Programming the System Timer (SysTick):

In the previous labs we used software-controlled delays. If accurate timing is required and/or interrupts vs. polling is required, the system timer in the processor can be used. Refer to Section 12.4 in the text for a detailed description of the System Timer.

There are three important control registers for the SysTick in Tiva: **STCTRL**, **STRELOAD**, **STCURRENT**.

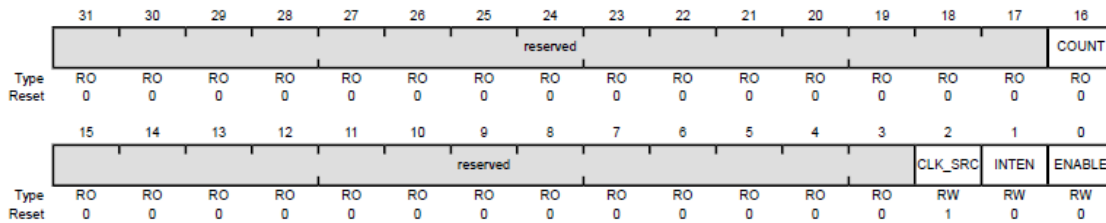
- The *Status and Control* register (**STCTRL** – Offset 0x010) is used to enable the SysTick system (interrupt vector address 0x0000.003C) (see Figure 8.1).
 - (**ENABLE - Bit 0**) This bit when set to 1 enables the SysTick system, and when set to 0, disables the SysTick system.
 - (**INTEN – Bit 1**) Enables interrupts for the SysTick system. When this bit is set to 1 interrupts are enabled, and the SysTick ISR will be called when the timer reaches 0.
 - (**CLK_SRC – Bit 2**) Clock Source bit. The internal oscillator of Tiva, which we have been using, is not the only clock available to use. With additional setup, another higher frequency oscillator can be used. However, for now we will use the quartz crystal based Precision Internal OSCillator (**PIOSC**) divided by 4. The PIOSC runs at 16 MHz on Tiva, so PIOSC/4 is 4 MHz. Hence the clock period is 250ns.

SysTick Control and Status Register (STCTRL)

Base 0xE000.E000

Offset 0x010

Type RW, reset 0x0000.0004

**Figure 8.1:** Status and Control register (STCTRL) at 0xE000.E010.

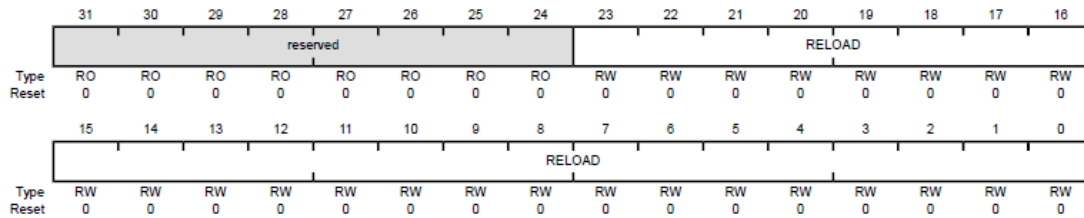
The *SysTick Reload Value* register (STRELOAD) below selects the time-out period of the SysTick system (see Figure 8.2). The 24-bit starting value for the counter is stored in RELOAD[23:0]. The SysTick system will count down from this value to 0, subtracting 1 every clock cycle.

SysTick Reload Value Register (STRELOAD)

Base 0xE000.E000

Offset 0x014

Type RW, reset -

**Figure 8.2:** SysTick Reload Value register (STRELOAD) at 0xE000.E014.

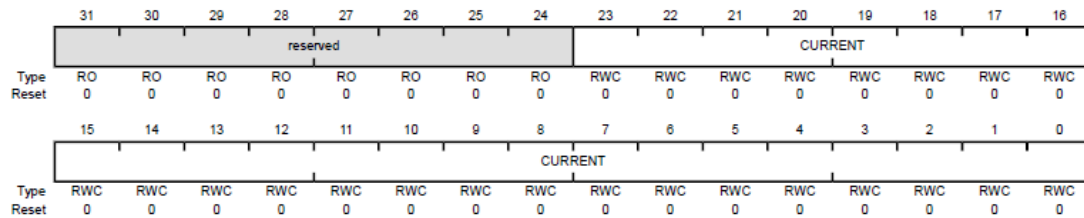
The *SysTick Current Value* register is a 24-bit register containing the current value of the count down from the RELOAD value.

SysTick Current Value Register (STCURRENT)

Base 0xE000.E000

Offset 0x018

Type RWC, reset -

**Figure 8.3:** SysTick Current Value register (STCURRENT) at 0xE000.E018

The *SYSPRI3* register configures the priority level, 0 to 7 of the SysTick exception handler by placing this level into the TICK field (bits 31:29). The lower the value of this field, the higher its priority. A priority level of 1 or greater is required to enable SysTick interrupts. This register is byte addressable.

System Handler Priority 3 (SYSPRI3)

Base 0xE000.E000

Offset 0xD20

Type RW, reset 0x0000.0000

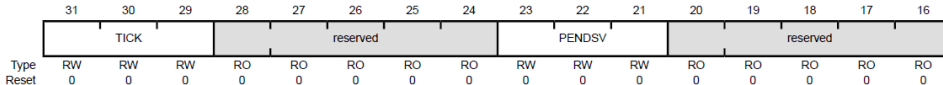
**Figure 8.4:** SYSPRI3 register at 0xE000.ED20

Figure 8.5 shows a program code that can be used to initialize and use the SysTick system.

- Lines 16-18 first disable the SysTick system during setup by clearing the ENABLE bit in the STCTRL register.
- Lines 19-21 load the number of clock cycles to count down from into the STRELOAD register.
- Lines 22-24 clear the CURRENT register by writing any value to it. Lines 25-27 set the priority of the SysTick to 2.
- Lines 28-30 enable the SysTick system and allow interrupts.
- Lines 41-46 is the SysTick Interrupt Service Routine (ISR). Note the .global directive on line 39 makes the ISR available to setup.s.
- The CPSIE I instruction on line 33 enables the maskable interrupts.

With this code, SysTick system will issue an interrupt about every second and will print “Hello from SysTick” on the screen.

```

1.      .equ NVIC_ST_CTRL,      0xE000E010
2.      .equ NVIC_ST_RELOAD,    0xE000E014
3.      .equ NVIC_ST_CURRENT,    0xE000E018
4.      .equ SHP_SYSPRI3,        0xE000ED20
5.      .equ RELOAD_VALUE,       0x003C0000
6.
7.          .global      OutStr
8.          .global      main
9.
10.     //*****
11.     // Program area including SysTick System Initialization
12.     //*****
13.          .section .text
14.          .syntax unified
15.          .thumb
16.  main:      LDR          R1, =NVIC_ST_CTRL
17.             MOV          R0, #0
18.             STR          R0, [R1]
19.             LDR          R1, =NVIC_ST_RELOAD
20.             LDR          R0, =RELOAD_VALUE
21.             STR          R0, [R1]
22.             LDR          R1, =NVIC_ST_CURRENT
23.             MOV          R0, #0
24.             STR          R0, [R1]
25.             LDR          R1, =SHP_SYSPRI3
26.             MOV          R0, #0x40000000
27.             STR          R0, [R1]
28.             LDR          R1, =NVIC_ST_CTRL
29.             MOV          R0, #0x03
30.             STR          R0, [R1]
31.     // SysTick is now set up. Main program follows.
32.     //
33.             CPSIE          I
34.  wait:      WFI
35.             B              wait
36.     //*****
37.     // SysTick ISR

```

```

38.      //*****
39.      .global sys_tick_handler
40.      .thumb_func
41.      sys_tick_handler:
42.          PUSH        {LR}
43.          LDR          R0,=hello
44.          BL           OutStr
45.          POP          {LR}
46.          BX           LR
47.
48.      hello:          .ascii      "Hello from SysTick\n\0"
49.
50.          .end

```

Figure 8.5: Program that initializes SysTick system and prints “Hello from SysTick” after each timeout period of 1s.

8.6 Procedure:

1. In the Lab 8 Canvas files an incomplete version of this alarm clock code is available for your use (Lab8.s). The code for the **setme** routine is functional but does not check for improper input values. The objective of this task¹ is to complete the digital alarm clock, including use of the Tiva LEDs that serves as a “silent” alarm indicator. After prompting the user to set the current time and the alarm, the clock will run until the board is reset. Once the alarm goes off (activating all 3 LEDs), it will stay on until any key is entered in the terminal. The alarm setting and current time should be displayed in the terminal emulation window each second as shown below. You can use the following steps for creating this digital alarm clock. [Show the working program to your TA.](#)
 - a) Write the **main** program. After initializing the LED alarm indicator port, **main** should prompt the user for the CLOCK setting followed by the ALARM setting (making calls to the **setme** routine to initialize both the clock and alarm variables). After initializing the clock and alarm variables, **main** should start the clock (by calling the **systick_ini** routine) and then remain in a loop waiting for the alarm and then for any key to be pressed (which, when pressed, should turn *off* the alarm LED).
 - b) Write the clock/alarm display routine **cdisp**. The **cdisp** routine should display the contents of the time sequence as “**hh:mm:ss XM**”, where **hh** is hours, **mm** is minutes, **ss** is seconds, and **X** is “A” or “P”.
 - c) Write the alarm monitor routine **almon**. This routine should compare the **time** and **alarm** sequences and, if an exact match occurs, the LED alarm indicator should be activated by setting “LedIsOn” to 1.
 - d) Write the LED alarm indicator port initialization routine **led_ini**. The LEDs are those on Port F on the Tiva board. See Lab 7 for how to configure Port F for this task. The LEDs should be off initially.
 - e) The clock update routine **clock** is provided. No corrections or improvements are needed.
 - f) Write the SysTick interrupt service routine **sys_tick_handler**. This straightforward routine should call the **clock** routine, the **almon** routine, and the **cdisp** routine.

¹ From the lab manual for EE362 course at Purdue University.

- g) Write the SysTick initialization routine **systick_ini**. This routine should enable the SysTick subsystem for a 1s interrupt rate and clear the “I” mask bit in the PRIMASK. This code is shown in Figure 8.5, but not as a subroutine. RELOAD_VALUE will also have to be corrected to correspond to exactly one second.
2. Modify the clock/alarm set routine **setme**. The **setme** routine prompts the user for entry of hours, followed by minutes, seconds, and either “A” or “P” (for AM or PM, respectively). Modify **setme** to assure the user provides a complete set of valid entries. You may choose not to check for valid data entries, but in that case, points will be deducted from your grade for this lab. **Show the working and modified program to your TA.**

8.7 Questions:

1. (5 pts) What are the differences between the initialization of the maskable and non-maskable interrupts?
2. (10 pts) List the similarities and differences of interrupt service routines over the subroutines.
3. (5 pts) Which delay routine is more accurate, the routine that uses the software program or the one that uses the SysTick system? Explain why.
4. (20 pts) EXTRA CREDIT: Consider the program written in Figure 8.6. This program makes use of the SysTick system to generate a 10 second delay. The program tries to show the ‘START’ message at the start of the program. Then it waits for 10 seconds before displaying the ‘STOP’ message on the screen. The program will not work, as there are errors in the program. Locate the errors in the program and modify the program so that it will work properly.

8.8 Lab report:

For the lab write-up, include

1. Programs that you wrote before the lab and a copy of your working .s files – 40 pts.
2. A brief discussion of the objectives of the lab and the procedures performed in the lab – 40 pts.
3. Answers to any **questions** in the discussion, procedure, or question sections of the lab – 20 pts.

Submit your files separately to Canvas, do not zip them together. Please keep in mind that you must have your work checked off (look for the green sections) by the TA to receive credit for the lab.

```

1.          .global SysTick_Init
2.          .global OutStr
3.
4.          .equ   RELOAD_VALUE,          0x000008FF
5.          .equ   NVIC_ST_CTRL,          0xE000E010
6.          .equ   NVIC_ST_RELOAD,        0xE000E014
7.          .equ   NVIC_ST_CURRENT,        0xE000E018
8.          .equ   SHP_SYSPRI3,            0xE000ED20
9.
10. //*****
11. // ROM area
12. //*****
13.          .section .rodata
14. starttxt: .ascii "Start\n\0"
15. stoptxt:  .ascii "Stop\n\0"
16. delsec:   .word  10
17.
18. ;*****
19. ; Program area
20. ;*****
21.          .section .text
22.          .syntax unified
23.          .thumb
24.          .global main
25. main:
26.          LDR    R1,=NVIC_ST_CTRL
27.          MOV     R0,#0
28.          STR     R0,[R1]
29.          LDR     R1,=NVIC_ST_RELOAD
30.          LDR     R0,=RELOAD_VALUE      // load reload val into R0
31.          STR     R0,[R1]
32.          LDR     R1,=NVIC_ST_CURRENT
33.          MOV     R0,#0
34.          STR     R0,[R1]
35.          LDR     R1,=SHP_SYSPRI3
36.          MOV     R0,#0x40000000
37.          STR     R0,[R1]
38.          LDR     R1,=NVIC_ST_CTRL
39.          MOV     R0,#0x01
40.          STR     R0,[R1]
41.          LDR     R0,=starttxt
42.          BL      OutStr
43.          LDR     R1,=delsec
44.          LDR     R2,=count
45.          LDR     R0,[R1]
46.          STR     R0,[R2]
47.          CPSIE   I
48. check:    LDR     R1,=count
49.          LDR     R0,[R1]
50.          CMP     R0,#0
51.          BNE     check
52.          LDR     R0,=stoptxt
53.          BL      OutStr
54. loop:     B      loop

```

```
55. //*****
56. // SysTick ISR
57. //*****
58.         .global sys_tick_handler
59.         .thumb_func
60. sys_tick_handler:
61.         LDR     R1,=count
62.         LDR     R0,[R1]
63.         SUBS    R0,R0,#1
64.         STR     R0,[R1]
65.         BX      LR
66. //*****
67. // RAM area
68. //*****
69.         .section .data
70.
71. count:      .space 1
72.
73.
74.         .end
```

Figure 8.6: Program that tries to implement a 10 second delay using SysTick system.